Joseph Perez & Hyber Gamboa

Professor Sendag

EGR 404

May 4, 2026

Final Report

https://github.com/Jperez67/RA-Shift-swap-Chat-bot-.git

https://gvievdeyvu5qtmhjr9ierq.streamlit.app/

The Problem

Managing Senior Residential Assistant (SRA) duty schedules is a tedious and often inefficient process. Traditional methods rely heavily on multiple static spreadsheets, manual updates, and informal communication between staff members that easily get lost within messages. These approaches frequently lead to issues such as scheduling conflicts, outdated information and missed updates.

The largest concerns with the duty schedule was managing shift swaps between RA's making sure they are eligible to do the shift swap, it is submitted in a timely manner, it is accurate among all spreadsheets, make sure all RA's know they are on call and need to get the on call phone. In doing this project the main goal was to design a system that would:

- Centralizes schedule data
- Automates shift swaps
- Maintains synchronization across platforms

- Provides an intuitive interface for users

Ultimately, the system aims to improve efficiency, reduce errors, and enhance usability through automation and intelligent interaction.

Methodology

To address the problem, the project was developed using an incremental, prototype-based approach. Instead of building the entire system at once, functionality was introduced in stages, allowing for testing and refinement at each step.

Prototype 1: Core Scheduling System

The initial implementation focused on building a reliable backend using Google Sheets and Apps Script. A structured data model was created with a central MasterSchedule and supporting sheets for swap requests and audit logs. Google Forms were used to submit swap requests, which were processed automatically through Apps Script triggers.

Prototype 2: Calendar Integration

The system was extended to integrate with Google Calendar. Each shift was represented as a calendar event, allowing schedules to be visualized and shared. This required storing event identifiers and updating events when schedule changes occurred.

Prototype 3: Roster Integration

A roster system was introduced to manage RA information, including email addresses and eligibility. This allowed for validation of swap requests and laid the groundwork for assigning users to calendar events.

Prototype 4: AI Interface and Frontend

The final stage introduced an AI-powered interface and a Streamlit web application. The AI system interprets natural language input, extracts structured data, and routes requests to the appropriate backend functions. A confirmation mechanism was added to ensure that actions such as swaps are explicitly approved before execution.

This phased approach ensured that each component was functional before integrating into a complete system. As well be able to revert back to previous prototypes if I wanted to take other directions later on in the process such as if I didn't like certain functions or the streamlit frontend that was used.

Tools and Technologies

I used the google ecosystem because I knew I could use app scripts to control the google scripts which google sheets is where the data currently resides for the calendars and info. I used Google Sheets as the primary data storage for the schedule, swap requests, Auditlogs, and roster data. This was then connected to the apps scripts which were used for all of the functions and back end logic. These functions consist of swap requests, validates data, synchronizes with google calendar. This at first was tied to gether in a simple system with a Google Form to be used as input for structured data

going in to the app scripts for changes to happen. The Google calender was used to provide visualization of the schedule and sends out invites to “Guests” who are the RAs on call that night so they get a notification to get the phone. Python(Streamlit) was used as the front end interface. It allows users to interact with the system through a chat based interface. OpenAI API Enables natural language understanding. It converts user input into structured data and determines user intent. Lastly GitHub Used for version control, organization of prototypes, and documentation.

Design Decisions

I used the Google ecosystem for ease of use and a seamless integration. The original data was already in Google Sheets and the use of appscripts made for quick easy working prototypes

Confirmation based actions because who is on call is very important and must be accurate so accidental removals or changes would not be okay so all changes must be confirmed when using the AI.

AI Integration and a Streamlit was important for me because then this became a project that I would be able to share with co workers where they put there own data and are able to easily use it with out being in the sheet manually calling functions it also create capability to send to RA’s so that they can send in regular chats to do shift swaps easily with out having to reach out to me. It truly wrapped the project nicely for public use.

Prototype Development was very important to me because then I could always have something to submit and show I had a working project but not only that if I wanted to revert to a simpler prototype I could just use that or even branch off in another direction if I didn't like the current state of the project in a later prototype.

Challenges and Problems Encountered

A big problem I had with the project is that since all of the back end is written in app scripts it is not very portable for someone to just open the git hub and run it it takes much more time to prepare and set up which is all covered in the setup.md file. The appscripts are also very slow at performing functions such where it could take a minute or two for one single shift swap where sometimes it works on the backend but the frontend has already timed out.

Some small problems included

Data Formatting Issues: User input from forms and AI often contained inconsistencies such as extra spaces or varying formats. This caused difficulties in matching records within the schedule. The issue was resolved by implementing normalization functions for strings and dates.

Trigger and Event Handling: Errors occurred when testing Apps Script triggers without proper event objects

Frontend–Backend Integration: Connecting Streamlit to Apps Script required deploying the script as a web application. Issues such as incorrect URLs, permissions, and JSON parsing errors had to be resolved through debugging and configuration

Results

The final system successfully integrates all components into a cohesive application. It provides:

- A centralized and structured scheduling system
- Automated and validated shift swaps
- Real-time synchronization with Google Calendar
- A natural language interface for interacting with the system
- A user-friendly frontend built with Streamlit

Users can:

- Ask schedule-related questions
- Request and confirm shift swaps
- Perform administrative actions
- View and manage scheduling data

The system significantly improves efficiency compared to manual scheduling methods and demonstrates the effectiveness of combining automation with AI.

.**Conclusion**

Overall the Project was a success with some valuable prototypes that can be used such as prototype 2,3 and 4. With 4 being the most complex incorporating AI. This is

something I will use in the future next year as a SRA to manage schedules. I would like to fine tune the front end page some more so that it is more seamless and would also like to find some more performance issues

If I were to do this again I would look into better front end applications and would look into ways I would work with this same project with out using app scripts which would hopefully run faster and be more portable to be shared.